Swapping Moshi's brain on a free T4: an honest feasibility report

Bottom line up front: replacing Moshi's Helium backbone with Phi-3.5 or Qwen2.5 and retraining on Colab free tier is not realistically achievable in a few weeks. Stock Moshi inference alone does not fit in a 15 GB T4, the backbone is tightly fused into Moshi's RQ-Transformer (not a plug-in), and the original training used **1,016 H100 GPUs** (Hugging Face +4) for what amounts to tens of thousands of H100-hours. The tractable version of this project is a hybrid: run a smaller Moshi-style system (Mini-Omni on Qwen2-0.5B) as a proof-of-concept backbone swap, and write a rigorous architectural analysis of why/how Moshi's Helium swap would work, grounded in the paper's exact numbers. This report walks through Moshi end-to-end, quantifies the architectural coupling, and lays out a three-week plan that produces a defensible course report without fantasy compute.

Part 1 — How Moshi actually works end-to-end

The audio pipeline begins at Mimi

When a user speaks, audio enters at **24 kHz mono**, is chunked into **80 ms frames (1,920 samples)**, and fed into **Mimi**, Kyutai's causal streaming neural codec. Mimi is a SeaNet/Encodec-style convolutional autoencoder (strides 4, 5, 6, 8, then an extra 1D stride 2) with an **8-layer Transformer bottleneck** (dim 512, 8 heads, MLP 2048, 250-frame context, RoPE). (Emergent Mind) The output is a latent tensor at **12.5 Hz** — one token-vector every 80 ms.

Each Mimi frame is quantized by an **8-level Residual Vector Quantizer** with codebook size **2,048** per level, yielding an overall bitrate of **1.1 kbps**. (Kyutai) Crucially, Mimi uses a **"split RVQ"**: the first codebook is a plain VQ distilled from **WavLM features** (it carries semantic content) while the remaining 7 are a standard RVQ chain carrying acoustic detail. (Emergent Mind) Outputs of the two branches are summed before decoding. (GitHub) Mimi is trained with **purely adversarial losses** (multi-scale STFT discriminator + feature matching), which pushed MUSHRA from 58.8 (with reconstruction losses) to **81.0** (Emergent Mind) — better than SpeechTokenizer at 4× the bitrate.

Dual streams and the inner monologue

Moshi models **two parallel audio streams** simultaneously: its own speech and the user's speech. (arXiv +5) For every 80 ms step there are therefore **16 audio codebook tokens** (8 Moshi + 8 user), plus **1 text token** from the "inner monologue" — a time-aligned transcript of what Moshi is saying, (Hugging Face +2) written in Helium's SentencePiece vocabulary. The joint sequence width is **K = 17 tokens per frame**.

The inner monologue is the paper's critical trick. (arXiv) Text tokens are inserted slightly **before** the audio tokens they describe (GitHub) (acoustic delay $\tau = 2$ frames during pretraining, $\tau = 1$ during fine-tuning). (Emergent Mind) Predicting text first forces linguistic planning to precede acoustic realization, which is why **Spoken QA tripled** over audio-only Moshi (LlamaQ rose from ~20% to **62.3%**). (Emergent Mind) By inverting the delay at inference, the same weights yield streaming ASR (**5.7% WER on LibriSpeech**) or streaming TTS (**4.7% WER, better than VALL-E's 5.9%**) with no retraining. (Emergent Mind)

The RQ-Transformer: Helium is the Temporal Transformer

Moshi's architecture is called **RQ-Transformer**. A large **Temporal Transformer** runs at 12.5 Hz across time, producing one hidden state per frame. That hidden state is then fed as a prefix into a much smaller **Depth Transformer** that autoregressively predicts the K=17 tokens *within* that frame (GitHub) (Substack) (first the text token, then 8 Moshi codebooks in RVQ order, then 8 user codebooks). The Depth Transformer has **6 layers, dim 1024, 16 heads, MLP 4096** — roughly 100 M parameters, versus the Temporal Transformer's 7 B. (GitHub) (GitHub)

Helium is the Temporal Transformer. The paper is explicit: Helium's weights are used to **initialize** the Temporal Transformer, (arXiv) (Kyutai) and its hyperparameters (**dim 4096, 32 layers, 32 heads, MLP 11264, RoPE, RMSNorm, SwiGLU, SentencePiece-32k, context 4096**) *define* the Temporal Transformer. This is not a "frozen LLM + adapter" design — once Moshi pretraining begins, Helium's weights are updated throughout all 1 M audio-training steps. The Depth Transformer, the audio embeddings (one 2048-entry embedding table per codebook), and the Mimi encoder/decoder are all trained jointly or already frozen. So **Helium is structurally fused in, not pluggable**.

Training stages and compute

Moshi's training proceeds through **five stages**, all on Kyutai's **Scaleway "Nabu 2023" cluster of 1,016 H100 GPUs across 127 DGX nodes**: (Hugging Face +2)

1. **Helium pretraining** — 500k steps, batch 4.2 M tokens, (Kyutai) ~**2.1 T English tokens**, (Substack) AdamW + cosine LR 3e-4, FSDP + activation checkpointing. (arXiv) (arXiv)
2. **Mimi codec training** — separate, adversarial-loss-only.
3. **Moshi audio pretraining** — 1 M steps on **~7 million hours** of unsupervised audio (arXiv) (transcribed by Whisper large-v3) as a single stream; half of every batch is text-only to preserve Helium's linguistic competence. (Kyutai)
4. **Multi-stream post-training** — 100 k steps, with two speakers faked by PyAnnote diarization. (Kyutai)
5. **Fisher + instruction tuning** — 10 k steps on ~2000 h Fisher (upsampled to 24 kHz), then 30 k steps on ~20,000 h of synthetic dialogues generated by Moshi's own streaming TTS. (Hugging Face +2) Moshiko (male) and Moshika (female) emerge here, each conditioned on a single voice actor across 70+ speaking styles to prevent impersonation. (Scaleway Labs)

Kyutai never publishes a single GPU-hour total, but the ballpark of 1,016 H100s running for multiple weeks across these stages implies **tens of thousands of H100-hours** — on the order of $25 k–$250 k in 2026 cloud prices.

Latency, streaming, and released artifacts

At inference, Mimi encodes 80 ms of audio into 8 codebook tokens, the Temporal Transformer consumes one frame, the Depth Transformer emits the next frame's 17 tokens, and Mimi decodes them. Theoretical latency is **160 ms** (Hugging Face +2) (two frames of buffer). (The Moonlight) Kyutai reports **~200 ms practical latency on an L4 GPU**, (Hugging Face) (GitHub) with **batch size 2 fitting in 24 GB**. (Gitbook) Full-duplex is achieved because the model always predicts *both* user-stream and Moshi-stream tokens every frame, so backchannels and interruptions fall out naturally. (Hugging Face) (MarkTechPost)

Released weights include **moshiko-pytorch-bf16** (15.4 GB safetensors), **moshika-pytorch-bf16**, experimental PyTorch q8 variants, MLX q4/q8/bf16 for Apple Silicon, and Rust/Candle bf16/q8 for the production server. The released Moshi weights are **CC-BY-4.0**, (GitHub +2) code is **MIT**. The **7B Helium that backs Moshi has never been released standalone**; the public Helium-1-2B and helium-1-preview-2B on HuggingFace are smaller, multilingual, Gemma-2-distilled models unrelated to Moshi's backbone except in name. (Hugging Face)

Part 2 — Feasibility of swapping Helium with Phi-3.5 or Qwen2.5

Helium is architecturally fused, not a drop-in component

Three properties of Moshi couple Helium deeply into the system. **First**, the Depth Transformer cross-attends (effectively) to the Temporal Transformer's hidden state of dimension **4096**; (Emergent Mind) swapping in Phi-3.5-mini (hidden 3072), (Emergent Mind) Qwen2.5-1.5B (hidden 1536), or Qwen2.5-3B (hidden 2048) forces a learned projection layer that is not in the codebase. **Second**, the inner-monologue text stream is tokenized by **Helium's SentencePiece unigram with 32,000 tokens**; (Substack) both Phi-3.5 (Llama tokenizer, 32,064 tokens, byte-level BPE) and Qwen2.5 (151,936-token tokenizer) use incompatible vocabularies. A different tokenizer means the text-stream embedding and output head must be resized and retrained, and *every existing alignment between audio and text in the 7 M hours of pretraining data becomes invalid*. **Third**, the audio codebook embeddings and their interleaving with text were learned end-to-end against Helium's specific representations. The Moshi paper's 5–10-point MMLU drop under 4-bit quantization (versus 2 points for Helium alone) shows that even weight-level perturbations of the Temporal Transformer materially damage the whole system. (Emergent Mind)

Spec comparison for candidate backbones

Model	Params	Hidden	Layers	Heads (Q/ KV)	Vocab	Context	Tokenizer	License
Helium (Moshi 7B)	7.0 B	4096	32	32/32 MHA	32,000	4096	SentencePiece	not released
Helium-1-2B (public)	2.02 B	2560	24	20/20 MHA	48,000	4096	SentencePiece	CC-BY-SA-4.0 (Gemma ToU)
Phi-3.5-mini-instruct	3.82 B	3072	32	32/32 MHA	32,064	128k LongRoPE	Llama BPE	MIT
Qwen2.5-0.5B	0.49 B	896	24	14/2 GQA	151,936	32k	Qwen BPE	Apache-2.0
Qwen2.5-1.5B	1.54 B	1536	28	12/2 GQA	151,936	32k	Qwen BPE	Apache-2.0
Qwen2.5-3B	3.09 B	2048	36	16/2 GQA	151,936	32k	Qwen BPE	Qwen Research

No candidate matches Helium's hidden dimension of 4096. The closest is Phi-3.5-mini at 3072, and it still requires a projection. Qwen2.5's GQA (only 2 KV heads) is architecturally efficient but changes the attention math the Depth Transformer's conditioning was trained against. None of these models share Helium's vocabulary.

What modifications would actually be required

At minimum, a swap entails: (1) a new text embedding table and LM head sized for the new tokenizer, (2) a learned linear projection from the new backbone's hidden dim to 4096 for the Depth Transformer, (3) remapping every training-data text transcript into the new tokenizer, (4) retraining the inner-monologue alignment from scratch — because the 80-ms text-to-audio timing was learned against Helium's subword granularity, and (5) adjusting the `LMModel` class in `moshi/models/lm.py`, the `get_moshi_lm` loader in `loaders.py`, and the Transformer module in `moshi/modules/transformer.py` to accept a different backbone. **The Moshi repo does not expose a backbone abstraction**; the Transformer is instantiated inline with Helium's shapes baked in. A clean swap means a non-trivial rewrite — a few hundred lines at minimum — not a config flag.

Training cost for the swap

Even if the code is rewritten, **full retraining is required**, not fine-tuning. The audio pretraining stage (1 M steps, 7 M hours, 1,016 H100s) is what teaches the Temporal Transformer to handle audio tokens at all; a new backbone has none of this. **A back-of-envelope calculation using 6·P·D FLOPs**: for a 3.8 B Phi-3.5 backbone and even 200 B audio tokens (an order of magnitude less than Moshi used), training cost is ≈ 4.6×10^{21} FLOPs. A T4 delivers ~65 TFLOPS effective in fp16, giving **~22 years of T4 wall-clock**. Even a 0.5 B Qwen2.5 backbone on 10 B tokens is ~3×10^{20} FLOPs ≈ **1.5 years of T4**. The Moshi paper provides no public GPU-hour total, but **the floor for any serious backbone retraining is clearly thousands of H100-hours** — infeasible on free compute by three or four orders of magnitude.

No one has published a Moshi backbone swap

A search of GitHub forks, HuggingFace, Papers with Code, and the arXiv/Reddit literature turns up **no community project that has swapped Moshi's Helium backbone**. Kyutai's own `moshi-finetune` repo supports only **LoRA fine-tuning** of the existing Moshi weights, GitHub and its README notes OOM issues even with LoRA at modest batch sizes on consumer GPUs. GitHub Related systems (LLaMA-Omni on Llama-3.1-8B, LLaMA-Omni2 on Qwen2.5-Instruct, GLM-4-Voice, Mini-Omni on Qwen2-0.5B) are separate architectures that happen to use different LLMs — they aren't drop-in replacements inside Moshi. The **absence of a community precedent is itself evidence** that the swap is harder than it looks.

Inference versus training on Colab free T4

Inference of stock Moshi does not fit. Kyutai's own README states plainly: *"At the moment, we do not support quantization for the PyTorch version, so you will need a GPU with a significant amount of memory (24GB)."* PyPI GitHub Moshi's bf16 weights alone are ~14–15 GB, and on a T4's 14.75 GiB usable VRAM Chris McCormick that leaves no room for activations, KV cache for minute-long dialogues, or Mimi. The practical demo runs on L4 (24 GB). Gitbook

The only CUDA-capable quantized variant is the Rust/Candle int8 build (~7 GB weights), but it requires building from source with `cargo build --features cuda` on Colab (~10 min first time, fragile across CUDA versions) and still has no guarantee of fitting once activations and KV cache are added. No GGUF, AWQ, or bitsandbytes community quantizations exist because Moshi's multi-stream generation loop is non-standard and unsupported by llama.cpp / vLLM. **Flash-Attention-2 does not support T4's Turing architecture**, GitHub Chris McCormick removing another memory lever. **Training on T4 is not feasible at any scale that would meaningfully update the backbone.**

Part 3 — Realistic project variants, ranked

Option A — full backbone swap + retraining + benchmark on Colab free: infeasible. The training FLOPs floor is thousands of H100-hours; Colab offers one T4 at ~1/30th H100 throughput with 12-hour session caps and no guarantee of GPU assignment. This option is a fantasy and the student should not promise it in a project proposal.

Option B — stock Moshi inference + documentation: partially feasible, fragile. The student can attempt the Rust/Candle int8 backend on Colab, accept that interactive mic demos are impossible (Colab cannot expose WebSocket servers or access microphones), and run file-based inference on pre-recorded WAVs. Budget two full days of debugging for Rust toolchain, CUDA version mismatch, and the Python-3.12 requirement of `rustymimi`/`moshi_mlx` GitHub (Colab ships 3.11). Deliverable is a characterization report, not a swap experiment.

Option C — architectural proposal + partial code scaffolding: feasible and rigorous. The student replaces the `Transformer` class in `moshi/models/lm.py` with a HuggingFace `AutoModelForCausalLM` wrapper around Phi-3.5-mini or Qwen2.5-1.5B, writes a projection layer from the new hidden dim to 4096 for the Depth Transformer, adjusts the tokenizer plumbing, and runs a **shape-only forward pass** on a dummy 4-second audio clip to show the tensors flow. No training happens — the model will produce garbage — but the code integrates and the report documents every dimensional mismatch and what retraining would cost. **This is a credible, honest contribution for a course project.**

Option D — tiny-scale demo: runs, but outputs incoherent audio. Swap Helium for Qwen2.5-0.5B, freeze Mimi and the Depth Transformer, attempt a few hundred LoRA steps on a small DailyTalk slice. Output will be unintelligible but gradients flow. Useful as an appendix or demo video, not a headline result.

Option E — use a smaller Moshi-like system as a real proxy: the most practical path. Mini-Omni (https://github.com/gpt-omni/mini-omni) is explicitly designed for adaptability, uses **Qwen2-0.5B** plus a Whisper-small audio adapter, arXiv totals ~1 B parameters, and fits comfortably on a T4. Its training code literally imports `Qwen2Model` from HuggingFace transformers; swapping it for SmolLM-1.7B, Phi-3.5-mini (int4), or Qwen2.5-1.5B is the kind of two-week project that actually produces results. The student can then write the Moshi-swap discussion analytically.

The recommended plan combines C and E: do the swap experiment on Mini-Omni (empirically), and write the Moshi-swap analysis with full architectural and compute accounting (analytically). This gives the student real numbers plus genuine engagement with the Moshi paper.

Part 4 — Concrete step-by-step plan for the recommended version

Project title and framing

"Swappable LLM backbones in streaming speech-to-speech models: an empirical study on Mini-Omni and an architectural analysis of Moshi." Framing the project this way is honest about what's empirically done versus analytically argued, and it still demonstrates deep engagement with Moshi specifically.

Stack and installation

On Colab, install PyTorch 2.3.1, Transformers 4.44, Accelerate, BitsAndBytes, PEFT, openai-whisper, pesq, pystoi, and the SpeechMOS/UTMOS package for automated MOS prediction. Clone Mini-Omni and install its requirements. For the analytical Moshi portion, clone `kyutai-labs/moshi` but do *not* attempt to run inference — instead, read `moshi/models/lm.py`, `loaders.py`, and `modules/transformer.py` and write unit tests that construct a replacement Temporal Transformer from HuggingFace weights.

Datasets

For the empirical Mini-Omni swap, use **DailyTalk** (14 GB, on HuggingFace as `kyutai/DailyTalkContiguous` GitHub or the original `keonlee9420/DailyTalk`) for LoRA fine-tuning after the swap, and a curated subset of **VoxEval-MMLU** or 20–50 spoken LibriSpeech utterances for evaluation. The full Fisher corpus is neither needed nor licensed for free use at this scale.

Evaluation methodology

Speech-to-speech evaluation spans five axes, arXiv and the student should report on four of them:

- **Intelligibility of synthesized speech**: transcribe the model's audio output with Whisper-small, compute **WER against the model's own inner-monologue text**. This cleanly isolates the TTS quality of the system from its reasoning.
- **Automated MOS**: run **UTMOS** on all generated clips for a no-humans-needed quality number. UTMOS correlates with human MOS on the order of 0.8 Spearman.
- **Latency**: measure wall-clock time-to-first-audio-chunk and real-time factor. This is easy and comparable across backbones.
- **Semantic coherence / reasoning**: TTS ~20 MMLU questions with Piper or bark-small, feed into the model, transcribe its answer with Whisper, grade against ground truth. This is the speech-LLM analog of MMLU accuracy.

Skip AudioBench (requires a Llama-3-70B judge on 80 GB GPU GitHub — impossible on free compute) **and full MUSHRA** (requires paid human raters). Optional: recruit 2–3 classmates for a small MOS rubric on 10 clips.

Whether to run stock Moshi as a baseline

No — do not promise a Moshi baseline, because stock Moshi PyTorch won't fit in 15 GB. If the student has 1–2 days to spare, they can attempt the Rust/Candle int8 build for a best-effort comparison, but the report should explicitly document the VRAM blocker and cite Kyutai's 24 GB requirement. Kyutai's demo at https://moshi.chat is a legitimate reference for qualitative behavior without needing local inference.

Known failure modes to budget for

Python-3.12-only wheels for `rustymimi` and `moshi_mlx` (GitHub) are incompatible with Colab's default 3.11. Colab idle-disconnects kill any training run longer than ~90 minutes of inactivity; students should use `keepalive` hacks or checkpoint every 5 minutes. GPU assignment denials during peak hours are frequent on free Colab — budget 20% schedule slack and have **Kaggle P100 (16 GB, 30 h/week)** as backup. Flash-Attention-2 does not support Turing (T4); stick with PyTorch SDPA. WebSocket inbound ports require `ngrok` or `cloudflared` on Colab, neither of which is officially supported for raw WS. **Plan for file-based I/O only.**

Week-by-week timeline

Week	Deliverables
1	Colab environment set up. Stock Mini-Omni inference runs on T4. Baseline 50-prompt evaluation recorded (WER, UTMOS, latency). Moshi paper §3 and §5 read and summarized. Architectural comparison table drafted.
2	Backbone-swap engineering: replace Mini-Omni's Qwen2-0.5B with Phi-3.5-mini-int4 or SmolLM-1.7B. Freeze audio adapters, LoRA-fine-tune ~100 DailyTalk clips for <30 min T4-time. Re-run the 50-prompt eval. Produce comparison plots.
3	Write analytical "What it would take to swap Moshi's Helium" section: cite paper hyperparameters and compute numbers, provide shape-test code for `LMModel` replacement, estimate retraining cost in H100-hours. Polish report (~10 pages), record 5-min demo video, prepare slides.

Part 5 — Honest unknowns and caveats

Several details the student may want are **not publicly documented**, and the report should flag them rather than guess:

- **Exact total GPU-hours of Moshi training.** The 1,016-H100 cluster is confirmed; Hugging Face +3 aggregate wall-clock and GPU-hours are not disclosed in the paper. Estimates of "tens of thousands of H100-hours" are order-of-magnitude inferences, not quoted figures.
- **Whether Kyutai plans backbone-swap support.** No public roadmap item addresses this; all evidence points the other way (the `moshi-finetune` repo is LoRA-only GitHub and the Helium dependency is hard-coded).
- **Whether Mimi's tokenizer is tied to Helium's embedding space.** Mimi is trained separately with adversarial losses on raw audio and can be used standalone (`kyutai/mimi` is a public checkpoint). Hugging Face Its RVQ codebooks are not conditioned on text. **Mimi itself is likely reusable across backbones**, but Moshi's *joint* text-audio embedding layer inside the Temporal Transformer almost certainly is Helium-specific.
- **Whether `moshiko-pytorch-q8` (experimental) or `moshiko-candle-q8` will fit on T4 in practice.** Kyutai publishes no VRAM numbers for these; the ~7 GB weight footprint suggests yes, but activations and KV cache may still push past 15 GB for long dialogues. The only way to know is to run `nvidia-smi` during a 30-second generation.
- **Whether Helium-1-2B (public) could substitute for the 7B Helium backbone.** It couldn't, trivially — its hidden dim is 2560 vs 4096, and it was distilled from Gemma-2 on only 24 EU languages Hugging Face Hugging Face rather than pretrained on 2.1 T English tokens. Substack The public Helium is essentially a different model with the same name.
- **Whether a clean forward pass shape-test would even succeed** on Moshi's codebase without subtle issues in the Depth Transformer's conditioning — nobody has published this.
- **Whether Whisper-WER against inner-monologue text is a fair intelligibility metric** across backbones, since different LLMs produce different text outputs; absolute WER comparisons across backbones are confounded by text-side reasoning quality. The student should report per-prompt deltas alongside absolute WERs.

Conclusion: the defensible project

Moshi is a beautifully integrated system precisely because it *isn't* modular — Helium, the Depth Transformer, Mimi's codebooks, and the inner-monologue delay pattern co-evolved (arXiv) through (arXiv) (Kyutai) 1 M audio-training steps on thousands of GPUs. That integration is what makes Moshi work at 200 ms latency (Hugging Face +2) on an L4, (GitHub) and it is also what makes a free-compute backbone swap fundamentally impossible. The student should not frame the project as "I will swap Moshi's backbone." They should frame it as "**Speech-to-speech models couple their LLMs tightly; here is what that coupling actually looks like in Moshi, and here is an empirical study of the analogous swap in Mini-Omni, which is small enough to run on a T4.**" That is a real contribution, it fits the compute budget, and it demonstrates exactly the kind of architectural literacy a Pattern Recognition and Neural Networks course should reward. The fantasy version of this project would earn a lower grade than the honest one.